Stack technique et sécurité — Application de cartes d'invitation

Oct 26, 2026 · @Djabirou

Stack technique retenue

La stack tient dans un seul projet web, avec une base de données et un stockage gérés par un service tiers. Moins de pièces, c'est moins de surface d'attaque et moins de maintenance.

Couche	Outil	Rôle
Interface et serveur	Next.js (React) + TypeScript	Pages, API, liens d'invitation rapides à ouvrir
Style	Tailwind CSS	Mise en page responsive
Éditeur de cartes	react-konva (Konva.js)	Glisser-déposer, redimensionnement, calques sur les modèles
Modèles	Fichiers JSON	Un modèle = une structure que l'éditeur dessine
Animation enveloppe	GSAP ou CSS 3D	Ouverture de l'enveloppe, sortie de la carte
Base de données	PostgreSQL via Supabase	Cartes, invités, réponses
Comptes	Supabase Auth	Connexion des organisateurs
Stockage photos	Supabase Storage	Photos de fond et photos de cartes
Compression photos	browser-image-compression	Réduit les images avant l'envoi
Hébergement	Vercel	Déploiement de l'application

Langage du serveur : en attente de ta réponse (JavaScript/TypeScript ou Python). Si tu choisis Python, le serveur passe en FastAPI et le reste ne change pas.

Menaces propres à l'application

Le risque numéro un du web reste le contrôle d'accès défaillant : c'est la catégorie A01 de l'OWASP Top 10:2025. Pour ton application, cela veut dire qu'un invité ou un inconnu ne doit jamais pouvoir lire ou modifier les cartes, les invités et les réponses d'un autre.

Menace	Où elle frappe	Conséquence	Défense principale
Contrôle d'accès défaillant	Cartes, invités, réponses	Un utilisateur lit ou modifie les données d'un autre	RLS en base + vérification de propriété dans chaque action serveur
Lien d'invitation devinable	URL de l'invité	Un inconnu ouvre des cartes privées ou répond à la place d'un invité	Code aléatoire de 16 caractères ou plus, jamais un numéro
Fichier malveillant	Envoi de la photo de fond	Fichier piégé, stockage saturé, contenu illégal hébergé chez toi	Types autorisés, vérification du contenu, taille maximale, nom généré
Fuite de données d'invités	Noms, numéros, réponses	Atteinte à la vie privée	Données minimales, suppression de carte en cascade, accès par rôle
Mauvaise configuration	Supabase, Vercel, Next.js	Table lisible par tous, clé secrète exposée	Revue de configuration avant chaque mise en ligne
Chaîne d'approvisionnement	Bibliothèques npm	Paquet compromis qui arrive dans ton code	Mises à jour, audit des dépendances, fichier de verrouillage
Abus et spam	Création de comptes, envoi de réponses	Coûts, pollution, déni de service	Limitation de débit sur les actions sensibles
Gestion d'erreurs	Messages d'erreur, pages de secours	Fuite d'informations internes, plantage	Messages génériques, journalisation côté serveur

Trois points sont spécifiques à ton architecture :

1. **Les invités ne se connectent pas.** Ils arrivent avec un lien. Leur accès repose donc entièrement sur le code du lien : il doit être long, aléatoire, et ne donner accès qu'à leur propre carte.

2. **Les actions serveur sont des points d'entrée publics.** Selon le guide OWASP sur Next.js, une action serveur reçoit des requêtes POST indépendamment de la page qui l'affiche : la protection de la page ne la protège pas.
3. **Une table Supabase sans RLS est lisible et modifiable par tout rôle qui a un droit dessus.** La documentation Supabase demande d'activer RLS sur chaque table exposée.

Contrôles de sécurité par couche

La règle de base est la défense en profondeur : aucune protection ne suffit seule, donc chaque couche vérifie à nouveau. Coche chaque ligne quand elle est faite et testée.

Accès des invités (le point le plus délicat)

- [ ] L'invité ne lit jamais les tables directement : une route serveur reçoit le code du lien, retrouve l'invité côté serveur et renvoie seulement son nom, sa carte et son statut
- [ ] Aucune politique `anon` avec `using (true)` sur les invités ou les réponses : cela donnerait la lecture de toutes les lignes à n'importe quel visiteur
- [ ] Code de lien aléatoire de 16 caractères ou plus, comparé côté serveur
- [ ] Limitation de débit sur la route qui reçoit les codes, pour freiner les essais en série

Base de données (Supabase)

Source : documentation Supabase sur RLS.

- [ ] RLS activé sur chaque table exposée : cartes, invités, réponses
- [ ] Droits retirés (`revoke all`) aux rôles `anon` et `authenticated`, puis redonnés uniquement pour les opérations utiles
- [ ] Une politique par opération (lecture, ajout, modification, suppression), avec `to authenticated` et `with check` pour qu'un utilisateur ne puisse pas réattribuer une ligne à un autre
- [ ] Un fichier de test par table, exécuté avec `supabase test db`, qui prouve les accès autorisés et refusés
- [ ] Vues créées avec `security_invoker = true`, sinon elles contournent RLS
- [ ] Aucune politique fondée sur `user_metadata`, que l'utilisateur peut modifier ; utiliser `app_metadata` pour les droits
- [ ] Clé secrète (`service_role`) uniquement côté serveur, jamais dans le navigateur
- [ ] Index sur les colonnes utilisées par les politiques, pour garder les requêtes rapides

Application (Next.js)

Source : guide OWASP Next.js.

- [ ] Autorisation vérifiée dans chaque action serveur et chaque route API, pas seulement dans le proxy ou le middleware (une faille, CVE-2025-29927, a permis de contourner une autorisation placée uniquement dans le middleware)
- [ ] Toutes les entrées traitées comme non fiables et validées par un schéma
- [ ] Identité de l'utilisateur déduite côté serveur, jamais lue dans les arguments envoyés par le navigateur
- [ ] Réponses réduites aux champs nécessaires (objets de transfert), jamais un enregistrement de base complet
- [ ] Préfixe `NEXT_PUBLIC_` réservé aux valeurs vraiment publiques, car elles se retrouvent dans le code envoyé au navigateur
- [ ] Limitation de débit sur les actions coûteuses : création de compte, envoi de réponse, envoi de photo
- [ ] `images.remotePatterns` précis, `serverActions.allowedOrigins` étroit, cartes sources du navigateur désactivées en production, jamais `next dev` en production
- [ ] Next.js maintenu à jour sur une version corrigée
- [ ] Tests négatifs : appeler les actions et routes directement, sans l'interface, avec un utilisateur A puis un utilisateur B

Photos envoyées

Source : aide-mémoire OWASP sur l'envoi de fichiers. OWASP rappelle qu'aucune technique ne suffit seule.

- [ ] Liste d'extensions autorisées réduite au strict besoin (par exemple JPG, PNG, WebP)
- [ ] Type de fichier vérifié par son contenu, car l'en-tête `Content-Type` peut être falsifié
- [ ] Nom de fichier remplacé par un identifiant généré par l'application
- [ ] Taille maximale et longueur de nom limitées
- [ ] Envoi réservé aux utilisateurs connectés
- [ ] Image décodée puis ré-encodée, avec suppression des métadonnées
- [ ] Stockage séparé de l'application (Supabase Storage), jamais dans le dossier public du serveur
- [ ] Pas de fichiers ZIP, un bouton pour signaler un contenu illégal, et un antivirus si l'offre le permet

Secrets, dépendances et surveillance

- [ ] Aucun secret dans le code ni dans le dépôt Git ; variables d'environnement uniquement
- [ ] Dépendances npm mises à jour régulièrement, audit avant chaque version (catégorie A03 de l'OWASP Top 10:2025)
- [ ] Journalisation des échecs de connexion, des refus d'accès et des erreurs, avec alertes (catégorie A09)
- [ ] Messages d'erreur génériques pour l'utilisateur, détails seulement dans les journaux (catégorie A10)
- [ ] Connexion protégée par une double vérification pour les organisateurs, si l'offre Supabase Auth le permet

Fluidité et performance

La fluidité se joue sur le téléphone de l'invité, pas sur ton ordinateur. Les trois premières lignes ci-dessous viennent de ton cahier des charges et de la documentation Supabase ; les autres sont mes recommandations, à vérifier par des mesures sur un téléphone modeste.

Sujet	Action	Effet attendu
Photos	Compresser et redimensionner dans le navigateur avant l'envoi (largeur minimale 1200 px, plafond de poids)	Envoi et chargement plus rapides
Base de données	Index sur chaque colonne utilisée par une politique RLS ; écrire `(select auth.uid())` plutôt que `auth.uid()`	Évite un parcours complet de la table à chaque requête (Supabase)
Cache	Ne jamais mettre en cache partagé une réponse propre à un invité ou à un organisateur	Évite qu'un invité voie la carte d'un autre (OWASP)
Animation	Animation 2D légère (GSAP ou CSS), sans modèle 3D lourd	Reste fluide sur les téléphones d'entrée de gamme
Ordre de chargement	Charger la photo de fond avant d'ouvrir l'enveloppe	Pas de fond vide pendant l'animation

Sujet	Action	Effet attendu
Page de secours	Afficher la carte directement si l'animation échoue ou si le navigateur est ancien	L'invité voit toujours sa carte
Test	Tester chaque version sur un téléphone d'entrée de gamme avec une connexion lente	Détecte les ralentissements avant les utilisateurs

Compétences et outils à maîtriser

Aucun « skill » Claude prêt à l'emploi ne sécurise une application à ta place : ce qui te protège, ce sont ces compétences, apprises dans cet ordre. Les trois premières couvrent l'essentiel des risques de ton projet.

Priorité	Compétence	Pourquoi pour ce projet	Où l'apprendre
1	Contrôle d'accès (autorisation)	Premier risque du web ; décide qui lit les cartes, invités et réponses	OWASP Top 10:2025, aide-mémoire Authorization
2	Row Level Security Postgres	Dernière barrière dans la base, même si le code a un défaut	Documentation Supabase RLS
3	Sécuriser Next.js (actions serveur, routes, données envoyées au navigateur)	Les actions serveur sont des points d'entrée publics	Guide OWASP Next.js
4	Validation des entrées	Texte des cartes, noms d'invités, réponses : tout vient d'inconnus	Aide-mémoire Input Validation
5	Sécurité des envois de fichiers	La photo de fond est ton principal point d'entrée de fichiers	Aide-mémoire File Upload
6	Tests de sécurité	Prouver que les accès refusés le sont vraiment	`supabase test db` (tests pgTAP) et tests directs des routes avec deux utilisateurs
7	Journalisation et alertes	Voir une attaque pendant qu'elle a lieu	Aide-mémoire Logging

Priorité	Compétence	Pourquoi pour ce projet	Où l'apprendre
8	Gestion des dépendances	Un paquet npm compromis arrive directement dans ton code	Aide-mémoire NPM Security

Pour la performance : mesurer sur un téléphone modeste (outils de développement du navigateur) et apprendre les bases de GSAP ou des animations CSS.

Limites : pourquoi « infaillible » n'existe pas

Aucune application n'est infaillible, et promettre le contraire à tes utilisateurs serait une faute. Même les cadres que tu suivras le disent : OWASP écrit qu'il n'existe pas de solution miracle pour valider le contenu envoyé par les utilisateurs, et qu'il faut empiler plusieurs protections. Des failles ont aussi touché Next.js lui-même, comme CVE-2025-29927, d'où l'importance de rester à jour.

L'objectif réaliste est donc double : rendre une attaque difficile, et limiter les dégâts quand elle réussit.

- **Sauvegardes** : copie régulière de la base, et test de restauration au moins une fois.
- **Plan de réponse** : savoir à l'avance qui change les clés, qui prévient les utilisateurs et qui coupe l'accès en cas de fuite.
- **Rotation des secrets** : pouvoir renouveler une clé en quelques minutes.
- **Données minimales** : ce que tu ne stockes pas ne peut pas fuiter ; ne garde que le nom, le nombre de places et la réponse.
- **Suppression** : effacer une carte supprime ses invités et leurs réponses.
- **Revue avant chaque mise en ligne** : parcourir la liste de contrôles de ce document.
- **Audit extérieur** : avant d'avoir de vrais utilisateurs payants, faire relire la sécurité par une personne expérimentée.

Texte à garder pour tes pages publiques : « Nous protégeons vos données avec les pratiques de sécurité actuelles », jamais « application 100 % sécurisée ».

Sources

Pages consultées le 2026-10-26. Les politiques et limites des offres gratuites changent : revérifie-les avant de t'engager.

- OWASP Top 10:2025 : liste des dix risques (A01 contrôle d'accès à A10 gestion des cas exceptionnels), lue dans les résultats de recherche

- OWASP File Upload Cheat Sheet : page ouverte et lue en entier
- OWASP Next.js Security Cheat Sheet : page ouverte et lue en entier
- Supabase, Row Level Security : page ouverte et lue en entier

Les lignes de performance qui ne citent pas de source sont des recommandations à confirmer par des mesures.